Arène attaquant / défenseur

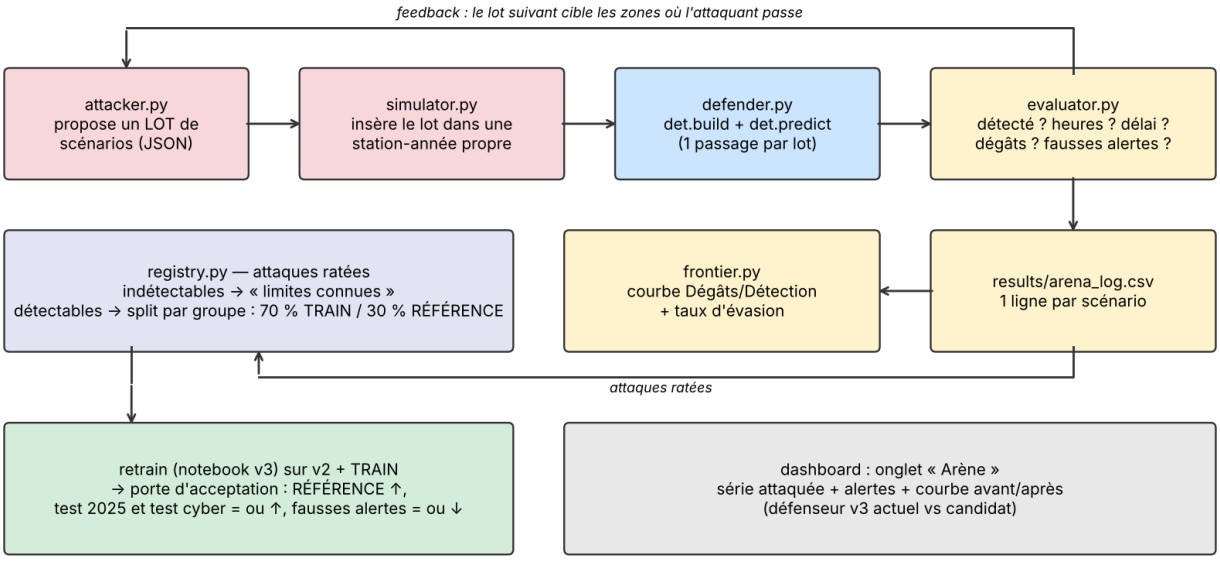
TSYP14 · Track 2 · Équipe IA · moteur + interface · rapport détaillé · 27/10/2026

En bref : un attaquant injecte de fausses données dans des stations USCRN (sur une copie des données propres), le détecteur v3 essaie de les attraper, chaque résultat est enregistré et les attaques ratées sont triées pour la suite. Une attaque complète (injection → témoins → score → décision → type d'attaque) prend ≈ **8 secondes**. Premier lot : **187 attaques en 2025, 35 % détectées** — l'arène montre en chiffres où le détecteur est fort (faux orages 75 %) et où il est faible (petits spikes 11 %, stealth 7 %). C'est une couche **au-dessus** du détecteur, pas un remplacement.

1. Ce qui se passe pour une attaque

Étape	Module	Ce qu'il fait
1 · Scénario	simulator.normalize	type, station, début, taille, durée, forme, capteurs falsifiés ; bornes par type (ex. spike ≤ 0,35, durée 1–12 h)
2 · Injection	simulator.inject	falsifie 5 cm / 10 cm / pluie sur une COPIE ; résolution réelle (0,001 m³/m³, 0,1 mm) ; hors de la plage observée de la station → refusé avec la raison ; décalage automatique (≤ 14 j) si une heure manque ; labels exacts
3 · Témoins	defender.FastDefender	~209 témoins du détecteur v3 ; jumeaux numériques appris une fois (dossier twins/) → ≈ 7 s par station au lieu de ≈ 75 s
4 · Score	detecteur_v3_final.joblib	étage 1 (horaire) + étage 2 (3 h → 7 jours) → probabilité ; alerte au seuil du modèle (47 %)
5 · Décision	engine.Arena	détecté ? délai, part des heures signalées, type d'attaque probable (classifieur 7 types), raisons en clair (témoins les plus rares)
6 · Registre	engine.Arena._registry	attaque ratée : SNR < 1 → « limite connue » (jamais apprise) ; sinon TRAIN (70 %) ou RÉFÉRENCE (30 %), tirés par GROUPE d'attaques

2. Architecture

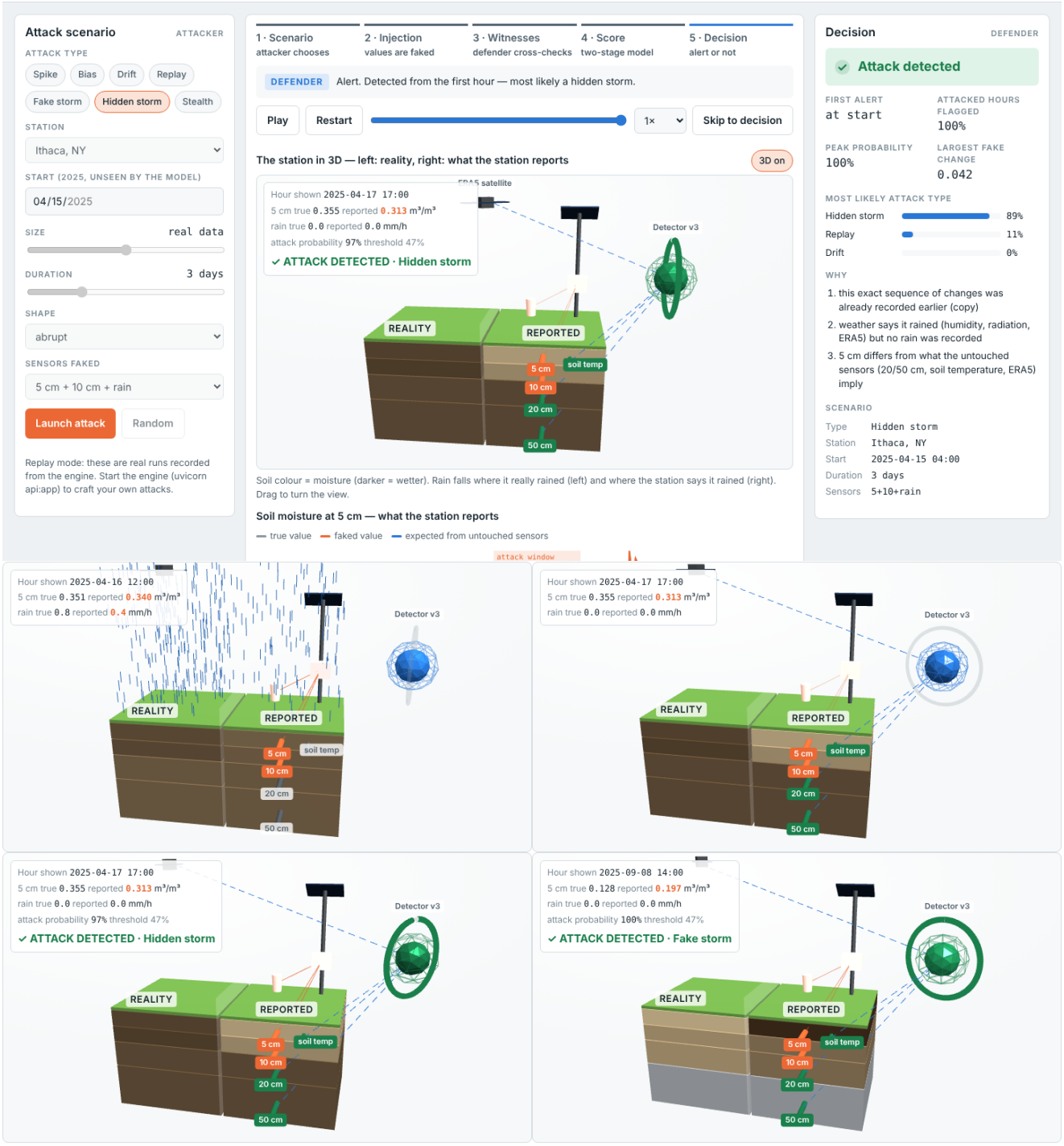


3. Validations

Contrôle	Résultat
Défenseur rapide = détecteur v3 complet ?	même calcul exact sur la base (écart 0,0) ; F1 2025 : seed1 0,599 / 0,615 · seed5 0,650 / 0,650 · seed12 0,623 / 0,635 · cyber 0,730 / 0,726
Simulateur	0 valeur hors résolution, 0 erreur de label, autres stations intactes ; rejets expliqués (plage, données manquantes, chevauchement)
Classifieur de type (test 2025)	bon type pour 70 % des attaques (hasard 14 %) : spike 100 %, faux orage 96 %, orage caché 76 %, dérive 65 %, replay 54 %, biais 48 %, stealth 38 % — biais, dérive et stealth se confondent
Rejouer une attaque d'un lot	identique (la source de l'orage / la courbe de séchage choisies sont enregistrées)
Paquet autonome	lancé seul (dossier neuf) : API, interface, une dérive détectée en 8,2 s

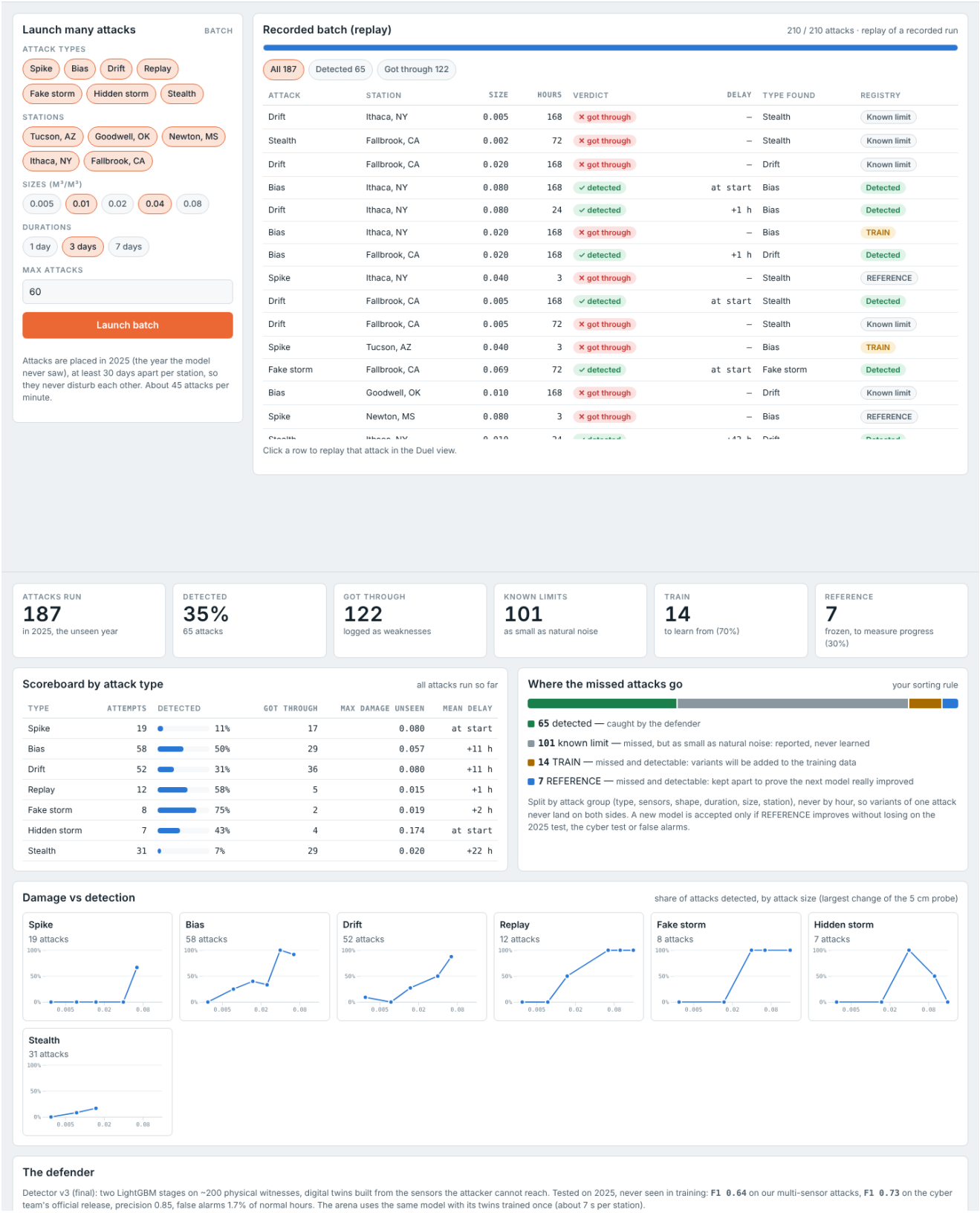
4. L'interface (3 vues)

Duel — une attaque animée en 5 étapes, avec une **scène 3D** de la station : bloc de sol coupé en deux (à gauche la réalité, à droite ce que la station déclare ; couleur = humidité), pluie réelle vs pluie déclarée, sondes falsifiées en rouge, capteurs non attaqués en vert qui envoient leurs « témoins » au détecteur, anneau = probabilité réelle, puis verdict. Sous la 3D : courbes 5 cm (vraie / falsifiée / attendue), pluie, capteurs non attaqués, bande des témoins, probabilité.



Les 4 moments de la scène 3D (orage caché à Ithaca) : injection (il pleut à gauche, moins à droite) → témoins → score 97 % → « ATTACK DETECTED · Hidden storm ».

Launcher — lots d'attaques (types, stations, tailles, durées) placés en 2025 à ≥ 30 jours d'écart par station ; progression en direct ; un clic sur une ligne rejoue l'attaque dans le Duel. **Dashboard** — chiffres clés, tableau par type, répartition du registre, courbes « dégâts vs détection » par type.



5. Premier lot (187 attaques acceptées, 2025)

Type	Essais	Détectées	Taux	Plus grand changement non vu (m³/m³)
Faux orage	8	6	75 %	0,019
Replay	12	7	58 %	0,015
Biais	58	29	50 %	0,057
Orage caché	7	3	43 %	0,174
Dérive	52	16	31 %	0,080
Spike (≤ 0,08)	19	2	11 %	0,080

Type	Essais	Détectées	Taux	Plus grand changement non vu (m³/m³)
Stealth	31	2	7 %	0,020

Le lot contenait beaucoup d'attaques volontairement minuscules (0,005–0,02) : ce taux n'est pas comparable au F1 du détecteur. Sur les 122 attaques ratées : **101 limites connues** (aussi petites que le bruit naturel, ≈ 0,015 m³/m³), **14 TRAIN**, **7 RÉFÉRENCE**.

6. Utiliser les attaques ratées : ce qu'on recommande

Cas	Correction
une famille passe mais est physiquement visible (ex. petits spikes)	ajouter un témoin ciblé — c'est ce qui a fait passer le détecteur de 0,45 à 0,67
détectable mais rare dans l'apprentissage	ajouter des variantes (TRAIN), placées en 2022–2024 uniquement
aussi petite que le bruit naturel	limite connue : on la rapporte, on ne l'apprend jamais

Un nouveau modèle n'est accepté que si : RÉFÉRENCE s'améliore, le test 2025 ne baisse pas, le test cyber (indépendant) ne baisse pas, les fausses alertes n'augmentent pas. **Jamais de réentraînement automatique** : notre propre générateur a déjà créé une fuite (0,73 qui valait 0,45). Risque principal : apprendre le style de notre générateur plutôt que celui d'un vrai attaquant.

7. Lancer l'arène

```
pip install -r requirements.txt          (garder scikit-learn 1.9.1 et LightGBM 4.7.0)
start_arena.bat                          (Windows : double-clic -> http://localhost:8000)
# sans serveur : ui/index.html via un serveur web -> mode « rejeu » (vraies exécutions enregistrées)
```

Route API	Usage
GET /api/meta	stations, types d'attaque, valeurs par défaut et bornes
POST /api/simulate	une attaque → toutes les images de l'animation
POST /api/batch · GET /api/jobs/{id}	lot en arrière-plan, progression, résultats
GET /api/scoreboard · /api/frontier · /api/log	données du tableau de bord (runs/arena_log.csv)
GET /api/replay/{job}/{attaque}	rejoue exactement une attaque d'un lot

8. Limites et suite

• Le défenseur travaille en mode lot (il regarde jusqu'à 7 jours après l'heure jugée) ; un mode « temps réel » avec score provisoire reste à faire.

• Les attaques viennent de notre générateur : un jeu indépendant de l'équipe cyber reste le juge final.

• Phase 2 possible : attaquant Optuna (recherche des attaques les plus « rentables »), une ronde de réentraînement validée par les 4 portes, carte des stations.